

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



FACULTY OF INFORMATION TECHNOLOGY

LAB REPORT

Lab 01: Xv6 User Programs

Instructor:

Leader:	Phuc-Khang Nguyen	24120068
Members:	Minh-Trong Pham	24120151
	Trong-Nghia Hoang	24120103

Ho Chi Minh City, Ngày 5 tháng 10 năm 2026

Table of Contents

1	Concurrent Prime Sieve (primes.c)	1
1.1	Ý tưởng thiết kế và Mô hình hoạt động	1
1.2	Khó khăn kỹ thuật và Giải pháp thực tế	2
2	Directory Tree Visualization (tree.c)	3
2.1	Ý tưởng thiết kế và Cơ chế Lookahead Buffer	3
2.2	Điểm sáng giải thuật và Xử lý ca biên	3
2.3	Kết quả thực nghiệm	4

1 Concurrent Prime Sieve (primes.c)

1.1 Ý tưởng thiết kế và Mô hình hoạt động

Chương trình hiện thực thuật toán Sàng Eratosthenes đồng thời (*Concurrent Prime Sieve*) dựa trên mô hình xử lý luồng dữ liệu qua chuỗi đường ống dẫn (*concurrent pipeline*).

Ý tưởng kết hợp đường ống Unix (*pipe*) với thuật toán sàng số nguyên tố xuất phát từ đề xuất mang tính nền tảng của Doug McIlroy — nhà khoa học máy tính tại Bell Labs và là người phát minh ra khái niệm Unix Pipeline [1, 2]. Mô hình này đại diện cho kiến trúc tính toán dòng dữ liệu CSP (*Communicating Sequential Processes*) được khởi xướng bởi C. A. R. Hoare [3], đồng thời vận dụng trực tiếp các cơ chế quản lý tiến trình và giao tiếp liên tiến trình (IPC) trong hệ điều hành xv6 [4, 5].

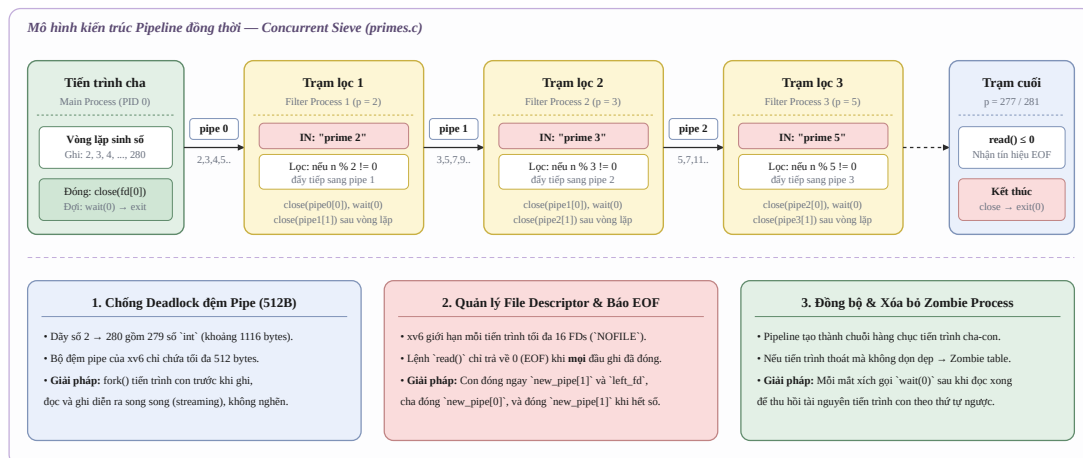


Figure 1: Mô hình luồng dữ liệu đường ống và các giải pháp hệ thống trong primes.c

Luồng thực thi diễn ra tuần tự qua các giai đoạn:

- **Tiến trình khởi tạo (Main Process):** Tạo `pipe 0`, thực hiện `fork()` để tạo tiến trình con lọc đầu tiên, sau đó ghi dãy số từ 2 đến 280 vào đầu ghi của `pipe 0`.
- **Các trạm lọc trung gian (Filter Stages):** Mỗi tiến trình đọc số nguyên đầu tiên p từ pipe bên trái (`left_fd`). Số p này chắc chắn là số nguyên tố và được in ra màn hình. Kế tiếp, tiến trình tạo một pipe mới (`new_pipe`), `fork()` tiến trình con kế tiếp, rồi đóng vai trò lọc: đọc các số n còn lại, nếu $n \pmod p \neq 0$ thì đẩy vào `new_pipe`.
- **Trạm kết thúc (Termination):** Khi đường ống bên trái đóng hoàn toàn, hàm `read()`

trả về giá trị ≤ 0 (EOF). Tiến trình thực hiện đóng đầu pipe, gọi `wait(0)` để đợi tiến trình con và thoát an toàn.

1.2 Khó khăn kỹ thuật và Giải pháp thực tế

Dựa trên các đặc thù kiến trúc và giới hạn tài nguyên của hệ điều hành xv6 [4], bài làm đã giải quyết ba thách thức then chốt:

1. **Ngăn ngừa Deadlock do bộ đệm Pipe giới hạn (512 bytes):** Dây số 2 $\rightarrow$ 280 tương ứng hơn 1.1 KB dữ liệu, vượt quá dung lượng đệm của một pipe trong xv6. Nếu tiến trình cha ghi toàn bộ dữ liệu trước khi con đọc, lời gọi `write()` sẽ bị chặn vô tận (*blocked*) gây treo hệ thống.

Giải pháp: Khởi tạo `fork()` trước khi bắt đầu ghi, giúp luồng dữ liệu được tiêu thụ đồng thời dạng streaming.

2. **Quản lý File Descriptor và tín hiệu ngắt EOF:** Hàm `read()` chỉ trả về 0 khi toàn bộ các đầu ghi của pipe đó được đóng. Nếu tiến trình con không đóng đầu ghi thừa kế thừa từ cha, lệnh `read()` sẽ chờ vĩnh viễn. Ngoài ra, xv6 giới hạn mỗi process chỉ có 16 FDs (`NOFILE`).

Giải pháp: Tiến trình con lập tức `close(new_pipe[1])` và `close(left_fd)`; tiến trình cha `close(new_pipe[0])` và `close(new_pipe[1])` ngay sau khi đẩy hết dữ liệu.

3. **Thu dọn tài nguyên tiến trình con (Zombie Process):** Tất cả các tiến trình cha trong chuỗi đều gọi lệnh `wait(0)` trước khi kết thúc, đảm bảo bảng tiến trình của hệ điều hành được dọn dẹp sạch sẽ theo khuyến nghị quản lý tiến trình [5].

2 Directory Tree Visualization (tree.c)

2.1 Ý tưởng thiết kế và Cơ chế Lookahead Buffer

Lệnh **tree** mô phỏng tiện ích dòng lệnh kinh điển trên hệ thống Unix/Linux do Steve Baker phát triển [6]. Chương trình duyệt hệ thống tệp đệ quy theo chiều sâu (DFS), hỗ trợ hiển thị cấu trúc phân nhánh đẹp mắt cùng các cờ tùy chọn **-L <depth>** (giới hạn độ sâu) và **-d** (chỉ hiển thị thư mục). Cấu trúc **struct dirent** và bảng chỉ mục **inum** được truy xuất trực tiếp từ hệ thống tệp tin xv6 [4].

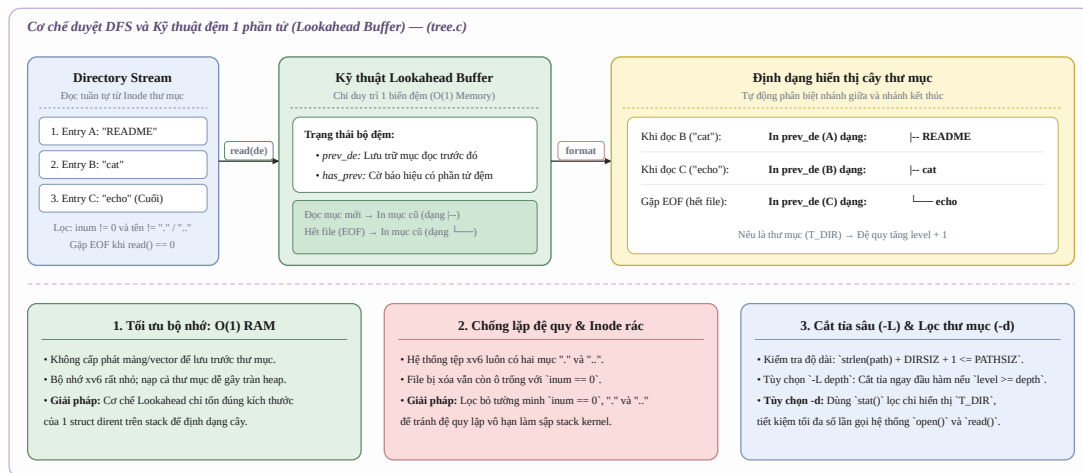


Figure 2: Cơ chế Lookahead Buffer $O(1)$ RAM và định dạng cây thư mục trong **tree.c**

2.2 Điểm sáng giải thuật và Xử lý ca biên

- Kỹ thuật đệm 1 phần tử (1-Item Lookahead Buffer) đạt $O(1)$ bộ nhớ:** Để vẽ đúng định dạng cây chuẩn Unix (phần tử giữa dùng |-, phần tử cuối dùng ___), phương pháp thông thường cần nạp toàn bộ danh sách tệp vào mảng/vector để biết chỉ số cuối cùng. Trong môi trường nhúng có bộ nhớ hạn hẹp như xv6, giải pháp này dễ dẫn đến tràn bộ nhớ heap.

Giải pháp: Sử dụng bộ đệm chỉ gồm 1 phần tử (`prev_de`, `prev_stat`). Khi đọc một entry hợp lệ mới từ thư mục, chương trình mới in entry trước đó bằng ký hiệu |-. Khi gặp EOF (hết thư mục), entry đang nằm trong bộ đệm chắc chắn là phần tử cuối cùng và được in bằng ___.

- Chống lặp đệ quy vô hạn và loại bỏ Inode rác:** Hệ thống tệp Unix luôn chứa hai

liên kết đặc biệt là "." và "..". Chương trình chủ động lọc bỏ hai mục này và các ô inode trống (`inum == 0`) để bảo vệ stack kernel không bị tràn.

3. Kiểm soát độ dài đường dẫn và cắt tỉa đệ quy:

- Kiểm tra an toàn trước khi ghép chuỗi: `strlen(path) + DIRSIZ + 1 <= PATHSIZ`.
- Tùy chọn `-L depth`: Dừng đệ quy sớm ngay từ điều kiện dừng (*base case*) `level >= max_depth`, tránh việc thực hiện lời gọi hệ thống `open()` không cần thiết.
- Tùy chọn `-d`: Dùng `stat()` kiểm tra kiểu inode (`T_DIR`), lọc bỏ tệp tin ngay trong vòng lặp đọc.

2.3 Kết quả thực nghiệm

Chương trình hoàn thành xuất sắc các ca kiểm thử: chạy mặc định `tree`, giới hạn độ sâu `tree -L 2`, lọc thư mục `tree -d`, và chỉ định đường dẫn cụ thể mà không gặp bất kỳ lỗi rò rỉ bộ nhớ hay cạn kiệt file descriptor.

References

- [1] M. D. McIlroy, “Coroutines: Semantics in terms of a hypothetical coroutine machine,” *Bell Laboratories Memorandum*, 1968.
- [2] M. D. McIlroy, E. N. Pinson, and B. A. Tague, “Unix time-sharing system: Foreword,” *The Bell System Technical Journal*, vol. 57, no. 6, pp. 1899–1904, 1978.
- [3] C. A. R. Hoare, “Communicating sequential processes,” *Communications of the ACM*, vol. 21, no. 8, pp. 666–677, 1978.
- [4] R. Cox, F. Kaashoek, and R. Morris, *xv6: a simple, Unix-like teaching operating system*, 2024. [Online]. Available: <https://pdos.csail.mit.edu/6.828/2024/xv6/book-riscv-rev4.pdf>
- [5] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.
- [6] S. Baker, *tree - list contents of directories in a tree-like format*, 1996, unix / Linux Manual Pages.